PROJECT REPORT

Metadata-Bound Digital Signatures for Document Approval

RSA-2048 · SHA-256 · PKCS#1 v1.5 · Metadata-bound TBS

Member: **Trần Công Hải**

Student ID: **23BA14094**

Single Project

November 2026

Table of Contents

1. Introduction 1
2. Cryptographic Design 1
    2.1 RSA-2048 1
    2.2 SHA-256 and EMSA-PKCS1-v1_5 1
    2.3 Metadata-bound TBS 1
3. System Architecture and Implementation 1
4. Verification and Failure Classification 2
5. Testing and Results 2
6. Security Analysis and Limitations 3
7. Conclusion 3
References 4

1. Introduction

Paper-based approval is being replaced by digital workflows, but a scanned signature or a password-protected PDF cannot prove that the approved bytes are the bytes that were reviewed. It also cannot bind the signer identity, original filename, file size and signing time to the document. This project implements a detached digital signature system for document approval. The signer produces a .sig package containing a PKCS#1 v1.5 signature over a canonical to-be-signed (TBS) structure; the verifier checks the document and every signed metadata field using only the signer's public key.

The cryptographic core is self-contained. Modular arithmetic, prime generation, RSA key generation, CRT signing, streaming SHA-256, EMSA-PKCS1-v1_5 encoding and signature verification are implemented in Python. The optional cryptography package is used only for encrypted private-key storage and interoperability tests. A command-line interface and a Tkinter desktop application expose the workflow, and the desktop application is packaged as a Windows executable.

2. Cryptographic Design

2.1 RSA-2048

Keys use a 2048-bit modulus and the public exponent e = 65537. Two probable primes p and q are generated with Miller-Rabin testing (40 rounds in production), and n = p·q. The private exponent is $d = e^{-1} \bmod \operatorname{lcm}(p-1, q-1)$; the CRT parameters dp, dq and qinv accelerate signing. Each signature is checked against the public key immediately after the CRT computation, so a faulty result is never released. The public-key fingerprint is SHA-256 over a domain-separated encoding of n and e.

2.2 SHA-256 and EMSA-PKCS1-v1_5

Documents are hashed with SHA-256 in 1 MiB chunks, so a large file is never loaded fully into memory. The TBS is hashed and encoded as an EMSA-PKCS1-v1_5 DigestInfo structure. The decoder validates the header, padding string, separator, DigestInfo prefix and digest. A signature whose length differs from the modulus, or whose representative is not smaller than n, is rejected before the public operation.

2.3 Metadata-bound TBS

The signature does not cover JSON whitespace or field order. It covers a canonical byte string rebuilt from the package metadata. A domain separator prevents cross-protocol reuse, and every variable-length field has a 32-bit length prefix, which removes concatenation ambiguities such as (signer="ab", name="c") versus (signer="a", name="bc").

```
TBS = b"DSIGv1\x00"
   || u32(len(algorithm)) || algorithm
   || u32(len(signer)) || signer
   || u32(len(original_name)) || original_name
   || u64(size)
   || u32(len(signed_at)) || signed_at
   || SHA-256(document)
```

Before signing, the signer identifier and filename are normalized to Unicode NFC; the timestamp must include an offset and is converted to UTC with second precision. Filenames must be basenames, so a package cannot carry path-traversal components.

3. System Architecture and Implementation

The code is divided into four layers: the cryptographic primitives, the domain models, the services that implement signing and verification, and the CLI/GUI interfaces. A .sig package is UTF-8 JSON with the following fields:

Field	Purpose	Validation
format / algorithm	Protocol and signature algorithm	DSIG/1 and RSA-PKCS1v15-SHA256
signer	Approver identity bound by the signature	NFC, 1–256 characters, no controls
key_fingerprint	Selects the public key	sha256: plus 64 lowercase hex
file.original_name	Original document name	Basename, 1–255 characters
file.size / sha256	Length and digest of the document	$0 \le size < 2^{64}$; 64 hex
signed_at	Claimed signing time (UTC)	ISO-8601 with offset, ≤ 64 chars
signature	Raw RSA signature	Non-empty base64url

Key files use a small JSON envelope. Private keys are written with mode 0600 and public keys with 0644; packages use 0644. Writes are atomic: a temporary file in the target directory is flushed, fsynced and renamed with os.replace. A private key can optionally be protected with PBKDF2-HMAC-SHA256 (200,000 iterations) and AES-256-GCM. The CLI provides keygen, sign, verify, inspect, fingerprint and tamper commands with JSON output and exit codes 0/1/2. The Tkinter application provides Keys, Sign, Verify and Tamper Demo tabs and runs long operations on a worker thread.

4. Verification and Failure Classification

Verification follows a fixed order so that the earliest possible cause is reported: parse the package; check the expected signer; compare the public-key fingerprint; confirm the file exists; compare its size; compute SHA-256; recover the signature and validate padding; rebuild the TBS and compare digests; report VALID. Comparisons use constant-time equality. Every failure maps to a stable reason code, which makes the CLI and GUI deterministic and testable.

Reason code	Meaning
PACKAGE_MALFORMED	Invalid JSON/UTF-8/schema, duplicate keys, non-finite numbers or oversized input
KEY_MISMATCH	Public key does not match the package fingerprint
SIGNER_MISMATCH	Signer differs from the expected identity
SIZE_MISMATCH / HASH_MISMATCH	Document length or SHA-256 differs from signed metadata
SIGNATURE_MALFORMED / PADDING_INVALID	Wrong signature length or invalid PKCS#1 v1.5 structure
TBS_MISMATCH	Signature does not match the signed metadata

Untrusted packages and key files pass through a strict JSON loader. It caps input at 1 MiB, rejects duplicate object keys and NaN/Infinity, limits integers to 4,096 digits, and rejects invalid UTF-8 and lone Unicode surrogates. Signer, filename and timestamp lengths are bounded. These checks are part of the hardening phase and prevent ambiguous parsing and resource-exhaustion inputs.

5. Testing and Results

The test suite contains 286 passing tests. It covers the cryptographic primitives, the package model, the services, the CLI, the GUI and the hardening layer.

Area	Checks	Result
Primitives	Miller-Rabin vectors, textbook RSA vector, CRT consistency, NIST SHA-256 vectors, PKCS#1 acceptance/rejection	Pass
End-to-end	Signing and verification, all 14 reason codes, metadata tampering, wrong keys, tampered signatures	Pass
Interfaces	CLI commands, JSON output, exit codes; GUI tabs, banners, key-context synchronization	Pass
Hardening	500 random JSON inputs, 200 random packages, 120 mutated packages, empty file, 50 MiB file, Unicode NFC/NFD	Pass
Interoperability	Byte-for-byte comparison with the cryptography package	Pass

A full GUI smoke test performed key generation, hashing, signing and verification (VALID), then created a one-bit tampered copy and observed HASH_MISMATCH. The 50 MiB streaming test confirms that large documents are hashed in chunks. The desktop application was packaged with PyInstaller 6.22.3

as a single-file Windows executable (16.18 MiB); the executable was launched successfully on Windows 11.

6. Security Analysis and Limitations

The implementation provides:

- Integrity of the document bytes and every signed metadata field.
- Authenticity for anyone holding the signer's trusted public key.
- Non-repudiation to the extent that the private key remains under the signer's control.
- Atomic persistence, restrictive private-key permissions and optional password encryption.
- Deterministic failure codes and strict parsing of untrusted metadata.

It does not provide:

- A public-key infrastructure, certificate authority, revocation or trust-on-first-use policy; public keys must be distributed and authenticated out of band.
- Trusted timestamping: signed_at is the signer's claimed time, not a third-party attestation.
- Protection against a compromised signer device or private key.
- Constant-time execution or protection against side-channel attacks on shared hardware.
- A replacement for a vetted production crypto library or standardized formats such as CMS and PAdES.

7. Conclusion

The project demonstrates a complete, auditable document-approval signature workflow: RSA-2048 key generation, SHA-256 hashing, strict PKCS#1 v1.5 signing, metadata-bound TBS construction, detached .sig packages, deterministic verification, a CLI, a desktop application and a Windows executable. The 286-test suite, fuzzing, large-file test and strict JSON policy provide practical evidence that the implementation behaves correctly on valid inputs and fails safely on malformed ones. Future work should add a PKI, hardware-backed key storage, trusted timestamps and standardized PDF signature profiles.

References

[1] R. L. Rivest, A. Shamir and L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems,” Communications of the ACM, vol. 21, no. 2, pp. 120–126, 1978.

[2] K. Moriarty, B. Kaliski, J. Jonsson and A. Rusch, “PKCS #1: RSA Cryptography Specifications Version 2.2,” RFC 8017, IETF, 2016.

[3] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015.

[4] National Institute of Standards and Technology, “Digital Signature Standard (DSS),” FIPS PUB 186-5, 2023.

[5] Python Software Foundation, “hashlib — Secure hashes and message digests,” Python 3 documentation, 2026.

[6] PyInstaller Development Team, “PyInstaller Manual,” version 6.22, 2026.

[7] Cryptography Developers, “cryptography: Python cryptography library documentation,” 2026.

[8] T. C. Hải, “DSIG: Metadata-Bound Document Approval Signatures,” source code repository, 2026. https://github.com/tranhai-shore/digital-signature-approval